

1. Number Systems & Conversion

- Value = $\sum b_i \cdot r^i$. Binary weights: 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024.
- n bits $\Rightarrow 2^n$ patterns, unsigned range 0 to $2^n - 1$.
- To store N distinct items: need $\lceil \log_2 N \rceil$ bits.
- Hex: 1 digit = 4 bits; Octal: 1 digit = 3 bits. **Group from the binary point**, pad far group with 0s. Hex: $A=10, B=11, C=12, D=13, E=14, F=15$.
- Dec $\rightarrow$ Bin (integer)**: divide by 2 repeatedly; remainders give bits **LSB first** (read bottom-up).
- Dec $\rightarrow$ Bin (fraction)**: multiply by 2 repeatedly; integer part popped out = next bit **after point** (top-down). May not terminate.
- Any base r** : divide by r (integer), multiply by r (fraction). Base $\rightarrow$ dec: $\sum \text{digit} \times \text{weight}$.

2. Signed Numbers

- Sign-magnitude**: MSB = sign, rest = magnitude. Two zeros.
- 1's complement**: negate = invert all bits. Two zeros.
- 2's complement**: negate = **invert all + 1**. One zero. **Used everywhere**.
- 2's comp value: MSB weight is -2^{n-1} , rest normal. Range -2^{n-1} to $+2^{n-1} - 1$ (one extra negative).
- 4-bit: $-8 \dots +7$; 8-bit: $-128 \dots +127$.
- Negate shortcut**: copy bits from right up to & including first 1, then invert all to its left.
- Sign extension**: copy MSB into new high bits ($-3 = 1101 = 11111101$).
- Most-negative number cannot be negated (gives itself).

3. Overflow Rules

- Unsigned ADD**: wrong $\Leftrightarrow$ carry-out $C_n = 1$.
- Unsigned SUB**: $(A + B' + 1)$: $C_n = 1 \Rightarrow A \geq B$ (no borrow, result correct); $C_n = 0 \Rightarrow A < B$ (bits are 2's comp of $B - A$). Carry = inverted borrow.
- Signed add/sub**: overflow $V = C_{n-1} \oplus C_n$ (carry into MSB XOR carry out).
- Adding a + and a - number **never** overflows.
- Bit-width detective**: wrong output = true value mod 2^n (re-read as signed if signed). Use a correct row for lower bound on n , a wrong row to pin 2^n .

4. Binary Codes

- BCD (8421)**: each decimal digit $\rightarrow$ 4 bits. Codes 1010–1111 (10–15) **never occur** $\Rightarrow$ don't-cares.
- Excess-3**: digit $n \rightarrow$ binary of $(n+3)$. **Self-complementing**: invert all bits of code for n gives code for $9-n$.
- Gray code**: consecutive values differ in **exactly 1 bit**, incl. wrap last $\rightarrow$ first. **Not weighted** (convert to binary to do arithmetic).
- Bin $\rightarrow$ Gray: keep MSB, $g_i = b_{i+1} \oplus b_i$ (parallel XORs).
- Gray $\rightarrow$ Bin: keep MSB, $b_i = b_{i+1} \oplus g_i$ (**cascaded**, MSB downward).
- Gray input in K-map**: write rows in physical (Gray) order; place each by its **binary value** as minterm #.
- Parity**: $P = d_3 \oplus d_2 \oplus d_1 \oplus d_0$ (even-parity gen). Checker = XOR of all received bits (0=ok). Catches any **odd** number of flips; misses even.

5. Boolean Algebra Laws

- Identity: $x+0 = x$, $x \cdot 1 = x$
- Null: $x+1 = 1$, $x \cdot 0 = 0$
- Idempotent: $x+x = x$, $x \cdot x = x$
- Complement: $x+x' = 1$, $x \cdot x' = 0$; Involution $(x')' = x$
- Commutative/Associative/Distributive: $x(y+z) = xy+xz$ and $x+yz = (x+y)(x+z)$
- Absorption: $x+xy = x$, $x(x+y) = x$
- Redundant literal: $x+x'y = x+y$, $x(x'+y) = xy$
- De Morgan: $(x+y)' = x'y'$, $(xy)' = x'+y'$ (break bar, flip operator)
- Consensus: $xy+x'z+yz = xy+xz$
- Precedence**: NOT > AND > OR. Duality: swap $\leftrightarrow$ and $0 \leftrightarrow 1$.
- Traps: $(AB)' = A'+B'$ (not $A'B'$); watch absorption e.g. $pqr+q+s = qs$.

6. Logic Gates (truth tables)

A	B	AND	OR	NAND	NOR	XOR	XNOR	NOT A
0	0	0	0	1	1	0	1	1
0	1	0	1	1	0	1	0	1
1	0	0	1	1	0	1	0	0
1	1	1	1	0	0	0	1	0

- $AND = 1$ iff all 1 (any 0 forces 0). $OR = 1$ iff any 1.
- $XOR = 1$ iff inputs differ; multi-input XOR = 1 iff **odd** number of 1s (parity). $XNOR = 1$ iff equal / even 1s.
- Controlling values**: $A \cdot 1 = A$, $A \cdot 0 = 0$ (enable/block); $A+0=A$, $A+1=1$; $A \oplus 0=A$, $A \oplus 1=A'$ (XOR = **programmable inverter**).

7. Minterms, Maxterms, Canonical Forms

- Row i : variable value as binary, **first-named variable** = **MSB**.
- Minterm m_i** : AND of all vars; variable = 0 $\rightarrow$ primed. Equals 1 only at row i .
- Maxterm M_i** : OR of all vars; variable = 1 $\rightarrow$ primed. Equals 0 only at row i . $M_i = m'_i$.
- Canonical SOP**: $F = \sum m(\text{rows where } F=1)$.
- Canonical POS**: $F = \prod M(\text{rows where } F=0)$.
- $\sum m$ list and $\prod M$ list are complementary (together cover all rows). $F' = \sum m(0\text{-rows})$.
- Missing var in a product term doubles the minterms it covers: $AB' = AB'(C+C')$.
- Standard forms**: SOP = ANDs into one OR (2-level). POS = ORs into one AND (2-level). ≤ 2 gate delays.

8. Karnaugh Maps

4-var cell numbering (rows AB , cols CD , Gray order 00, 01, 11, 10):

$AB \backslash CD$	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

- Core fact: two minterms differing in 1 variable merge, that variable vanishes. Group of 2^k cells removes k variables.
- Group rules**: rectangle of 1, 2, 4, 8, 16; may wrap edges & 4 corners; only 1s (and chosen Xs); as **large** and as **few** as possible.
- Reading a group: vars constant over it survive (0 $\rightarrow$ primed, 1 $\rightarrow$ plain); vars that change vanish.
- PI**: group that cannot be enlarged. **EPI**: PI covering a 1 that **no other** PI covers (must be used).
- Procedure**: (1) find all PIs; (2) take EPIs; (3) cover remaining 1s with fewest largest PIs.
- Don't-cares (X)**: treat as 1 only if it enlarges a group; each X decided independently; never make a group of only Xs; Xs never make a PI essential.
- POS from K-map**: group the **0s** $\rightarrow$ minimal SOP of $F' \rightarrow$ De Morgan each term. With don't-cares, MSP and MPS may differ as functions.
- Cyclic map**: no EPIs $\Rightarrow$ several equal minimal answers.
- 4 corners $\{0, 2, 8, 10\} = B'D'$.
- 5-var**: two 4-var maps ($A=0, A=1$); a cell is also adjacent to same cell in other map; group in same place in both $\Rightarrow$ loses A .

9. Quine–McCluskey

- List minterms and **don't-cares** in binary, grouped by number of 1s.
- Merge terms from adjacent groups differing in 1 bit $\rightarrow$ put '–' at that position; tick both. Only merge terms with dashes in same places.
- Repeat until no merges. **Unticked terms = prime implicants**.
- PI chart**: rows=PIs, cols = required minterms only (**exclude don't-cares**). Column with single $\times \Rightarrow$ that PI is essential.
- Dash string to literal: dash=var gone, 1=plain, 0=primed.

10. NAND / NOR Implementation

- Both universal. **From NAND**: NOT=1, AND=2, OR=3 (OR=1 if complements free), XOR=4. **From NOR**: NOT=1, OR=2, AND=3, XNOR=4.
- NOT: tie inputs. $AND = ((AB)')'$. $OR = (A'B')'$ (De Morgan).
- SOP $\rightarrow$ **NAND-NAND**: add 2 bubbles on each middle wire; $AND \rightarrow NAND$, $OR(\text{bubbled inputs}) \rightarrow NAND$. Same gate count as $AND-OR$.
- POS $\rightarrow$ NOR-NOR**: dual; $OR \rightarrow NOR$, $AND(\text{bubbled}) \rightarrow NOR$.
- Single-literal term**: needs its complement at 2nd level (feed literal' or add inverter).
- 2-input gates only**: get minimal SOP/POS, **factor** to cut fan-in & share subterms, then push bubbles, then count.
- Choose: $NAND \rightarrow$ use minimal **SOP**; $NOR \rightarrow$ use minimal **POS**.

11. Half / Full Adder

Full Adder				
A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

- Half adder**: $S = A \oplus B$, $C = AB$.
- Full adder**: $S = A \oplus B \oplus C_{in}$; $C_{out} = AB + AC_{in} + BC_{in} = AB + C_{in}(A \oplus B)$ (majority).
- S K-map is checkerboard (no simplification) $\Rightarrow$ XOR gates.

- **FA = 2 HA + OR:** HA1(A, B) $\rightarrow (s_1, c_1)$; HA2(s_1, C_{in}) $\rightarrow (S, c_2)$; $C_{out} = c_1 + c_2$ (both carries never 1 together, OR ok).
- **Ripple-carry:** n FAs chained, carry to carry. Delay = $2n+1$ gate levels ($O(n)$). 64-bit = 129 delays.
- **FA as gate:** $C_{in}=0$: $S=A \oplus B$, $C=AB$. $C_{in}=1$: $S=(A \oplus B)'$, $C=A+B$.

12. Half / Full Subtractor

Half Sub ($X-Y$)			
X	Y	B(borrow)	D(diff)
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

- **Half sub:** $D = X \oplus Y$, $B = X'Y$ (not symmetric: $X'Y \neq Y'X$).
- **Full sub:** $D = X \oplus Y \oplus B_{in}$; $B_{out} = X'Y + X'B_{in} + YB_{in} = X'Y + (X' + Y)B_{in}$.
- **FS = 2 HS + OR:** HS2 must compute ($d_1 - B_{in}$) i.e. ports $.x(d_1)$, $.y(B_{in})$ — swapping them is the classic bug.
- **HS as gate:** $X=1$: $D=Y'$ (inverter), $B=0$. **FS as gate:** $X=0$: $B_{out}=Y+B_{in}$; $X=1$: $B_{out}=Y \cdot B_{in}$.

13. Adder/Subtractor & Carry Lookahead

- **Add/Sub:** XOR every B_i with mode M ; connect M to C_0 . $M=0$ add, $M=1$ subtract ($A + B' + 1$). Overflow $V = C_{n-1} \oplus C_n$.
- **CLA:** generate $G_i = A_i B_i$; propagate $P_i = A_i \oplus B_i$. $C_{i+1} = G_i + P_i C_i$; $S_i = P_i \oplus C_i$.
- $C_1 = G_0 + P_0 C_0$; $C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$; $C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$.
- Delay = constant **4 gate levels** (any n); gate count $O(n^2)$; fan-in grows. Hierarchical lookahead $\rightarrow O(\log n)$ delay, $O(n)$ gates.
- For carry, P may be $A+B$; for sum it must be $A \oplus B$.
- **Borrow lookahead:** $G_i = X'_i Y_i$, $P_i = X'_i + Y_i$ (OR form).

14. BCD Adder & Code Converters

- **BCD adder:** add two BCD digits + C_{in} with 4-bit binary adder (sum $K Z_8 Z_4 Z_2 Z_1$). If sum > 9 add 0110 and set decimal carry.
- Correction needed $C = K + Z_8 Z_4 + Z_8 Z_2$. Feed $0 C C 0$ to 2nd adder; C = decimal carry out.
- n -digit = n such stages, carry to carry.
- **BCD $\rightarrow$ Excess-3:** 4 in, 4 out, don't-cares 10–15; one K-map per output: $z = D'$, $y = (C \oplus D)'$, etc.
- **Multi-output design:** count I/O; truth table with Xs for impossible inputs; one K-map/output; share gates.

15. Multipliers & Comparator

- Partial product = multiplicand AND one multiplier bit, shifted by its position; then add rows.
- 2×2 : $C_0 = A_0 B_0$; C_1 =HA of $A_1 B_0, A_0 B_1$; C_2 =HA of $A_1 B_1$ & carry; C_3 =its carry.
- $J \times K$: $J \cdot K$ AND gates, $(J-1)$ K -bit adders, product $J+K$ bits.
- Multiply by const: $\times 6 = \times 4 + \times 2$ = (shift-left-2)+(shift-left-1); one adder per extra 1 in constant.
- **Comparator:** equal bit $x_i = (A_i \oplus B_i)'$ (XNOR). $A=B$: $x_3 x_2 x_1 x_0$.
- $A > B = A_3 B'_3 + x_3 A_2 B'_2 + x_3 x_2 A_1 B'_1 + x_3 x_2 x_1 A_0 B'_0$ (dictionary order from MSB); $A < B$ symmetric.

16. Decoders

- n inputs $\rightarrow 2^n$ outputs, exactly one active. 2^n AND gates. Output $D_k = m_k$.
- **Active-high:** outputs = minterms. **Active-low:** (bubbles) = maxterms M_k .
- **Enable:** $EN=0 \Rightarrow$ all outputs off. Decoder+enable = **demux** (data on EN).
- **Build big:** MSB(s) drive enables of smaller decoders (3-to-8 = two 2-to-4, A'/A enable).
- **Implement F :** active-high + OR of 1-rows; active-high + NOR of 0-rows; active-low + NAND of 1-rows; active-low + AND of 0-rows. Choose gate side with fewer rows = $\min(\#1s, \#0s)$.

- One decoder serves many functions (own OR gate each). A single needed minterm needs no gate.

17. Encoders

- 2^n in $\rightarrow n$ -bit index of active line. Plain encoder = OR gates: $Y_2 = D_4 + D_5 + D_6 + D_7$, $Y_1 = D_2 + D_3 + D_6 + D_7$, $Y_0 = D_1 + D_3 + D_5 + D_7$.
- Fails: two inputs active $\Rightarrow$ OR of codes (garbage); none active = D_0 active (both give 0).
- **Priority encoder:** highest active wins + **Valid** bit. 4-to-2: $x = D_2 + D_3$, $y = D_3 + D_1 D'_2$, $V = D_0 + D_1 + D_2 + D_3$.
- **Decoder $\rightarrow$ encoder:** wire decoder line k to encoder input $f(k)$ = any 1-to-1 code converter (must be one-to-one).

18. Multiplexers & Demux

- 2^n data, n selects. $Y = \sum_k m_k(S) \cdot I_k$. 2:1: $Y = S'I_0 + SI_1$. 4:1: $Y = S'_1 S'_0 I_0 + S'_1 S_0 I_1 + S_1 S'_0 I_2 + S_1 S_0 I_3$.
- Enable (often active-low bubble): disabled $\Rightarrow$ output 0.
- Mux with data 0, 1, 1, 0 = XOR gate.
- **Implement F , n vars on n selects:** tie each I_k to that truth-table row's output ($0/1$).
- $n-1$ **on selects:** each data input $\in \{0, 1, x, x'\}$ via implementation table (rows x' and x , circle 1s, read each column).
- $n-2$ **on selects:** each data input = function of the 2 leftover vars (K-map the column, use Xs).
- “ AB as $s_1 s_0$ ” $\Rightarrow A$ = MSB select. Order matters.
- Only-true-inputs trick: feed leftover vars to a decoder = free complemented minterms.
- **Demux:** 1 data $\rightarrow$ one of 2^n outputs = decoder with EN as data.

19. PLDs (ROM / PLA / PAL)

- All = AND array (products) feeding OR array (sums). ‘ $\times$ ’ on a cross = connection.
- **ROM:** AND fixed (full decoder, all 2^n minterms) + OR programmable. = stored truth table. Size $2^n \times m$.
- **PLA:** both arrays programmable; a product term shared across outputs. Size = inputs $\times$ #product terms $\times$ outputs. “Min size” = **fewest distinct product terms** (share across outputs, even non-prime terms).
- **PAL:** AND programmable, OR fixed (each output has own ANDs). Minimise each output separately.
- **Programming table:** per product term: input col 1=true literal, 0=complemented, –=unused; output col 1=connected.

20. Verilog Essentials

- Gate primitive: **output first** — **and(y,a,b)**. Structural/**assign** are concurrent; line order irrelevant.
- **wire:** driven continuously (gate out, **assign**, module output); cannot be set in **initial/always**.
- **reg:** holds last value; assigned in **initial/always**. Not necessarily hardware register.
- Testbench: DUT inputs = **reg**, DUT outputs = **wire**.
- Named ports .p(sig) safe; positional must match order (bug risk for non-symmetric blocks).
- Sequential **always @ (posedge clk)** use **<=** (non-blocking: eval all RHS, then update). Combinational **always @ (*)** use **=**; assign every output on every path else a **latch** is inferred.
- Blocking = updates immediately; non-blocking **<=** pending till end of step. Arithmetic wraps to dest width.
- **^a** bitwise, **!a** logical; **&a** reduction-AND, **^a** parity. **assign ys=i1:i0 = 2:1 mux**. {c,s}=a+b+cin catches carry.
- **\$display** once; **\$monitor** on any change (end of step); **\$time**; **\$finish** end, **\$stop** pause. **#n** delay.

Exam Quick-fire

- Always write truth table **before** K-map, mark don't-cares.
- Label K-map axes with variable names + Gray order; recheck which letter is MSB.
- “Only these components”: first write what each allowed block computes, then map pieces.
- State tables / listed answers: use the exact column order & notation the question gives. Marking is often all-or-none.